

TDE – MODELAGEM E SEGURANÇA DE BANCO DE DADOS

Daniel Ribeiro Gonçalves – 00021515

Pedro Otávio Silva De Oliveira – 00021165

1. Descrição do projeto

O projeto consiste no desenvolvimento e administração do banco de dados para uma rede social de microblogging, semelhante ao Twitter. O foco desta etapa inicial não é a modelagem completa de todas as entidades possíveis, mas sim o planejamento da administração, da segurança e do controle de acesso das funcionalidades principais que serão entregues aos usuários. A plataforma permitirá o compartilhamento de mensagens rápidas (posts), focando na interação dinâmica, mas com um controle rigoroso do que cada perfil pode acessar ou modificar na base de dados.

2. Descrição do sistema

O sistema vai ser uma aplicação cliente-servidor focada em alta disponibilidade e resposta rápida. A interface do usuário pode ser desenvolvida utilizando Flutter e Dart para garantir uma experiência mobile responsiva e nativa, comunicando-se através de uma api REST estruturada em Python (utilizando o framework Flask). Toda regra de segurança de dados não ficará apenas na aplicação, mas será refletida diretamente no SGBD, garantindo que a visualização e manipulação de informações sejam estritamente limitadas aos níveis de privilégio definidos no banco

3. Análise de requisitos

1. Requisitos Funcionais (RF)

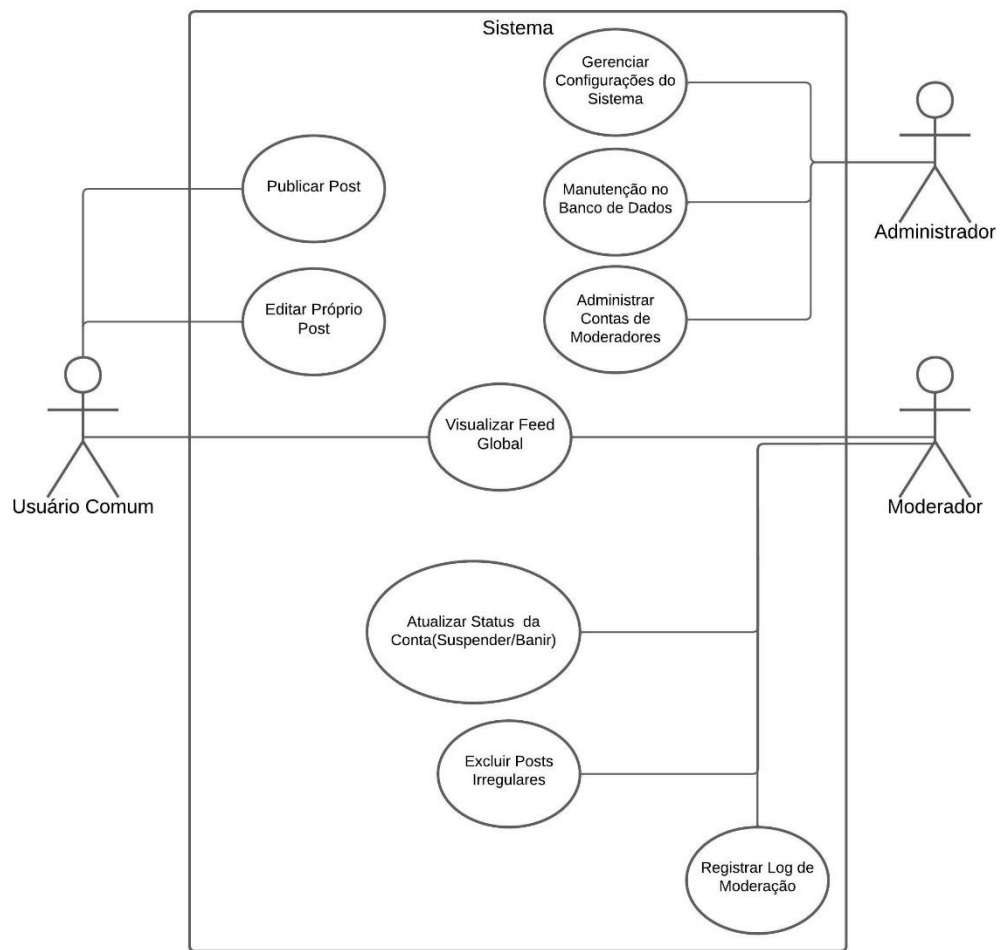
- **RF01 - Gestão de Postagens:** O sistema deve permitir que usuários comuns insiram novos posts de texto curto e visualizem um *feed* cronológico com as postagens de todos os usuários públicos.

- **RF02 - Edição Restrita:** O sistema deve permitir que um usuário altere o conteúdo de suas próprias postagens, sendo vetada a alteração de postagens de terceiros.
- **RF03 - Moderação de Conteúdo:** O sistema deve fornecer aos moderadores a capacidade de visualizar todas as postagens e excluir aquelas que violem as diretrizes da comunidade.
- **RF04 - Gestão de Contas:** Moderadores devem ser capazes de alterar o status das contas de usuários (ex: ativo, suspenso, banido), enquanto a administração geral de permissões e infraestrutura caberá exclusivamente aos Administradores (DBAs).

2. Requisitos Não Funcionais (RNF)

- **RNF01 - Segurança em Nível de SGBD:** As regras de negócio relacionadas a acesso não devem residir apenas no código da aplicação (ex: *Python/Flask*). O controle de privilégios de inserção, leitura, atualização e exclusão deve ser mapeado diretamente em *Roles* no banco de dados, justificando os níveis de privilégio de cada perfil.
- **RNF02 - Integridade e Auditoria:** O banco de dados deve possuir mecanismos (como tabelas de log e restrições de deleção) para garantir a rastreabilidade. Ações de moderação (como a exclusão de um post) devem gerar um registro em uma tabela de auditoria restrita, impedindo que moderadores apaguem o próprio rastro.
- **RNF03 - Disponibilidade e Desempenho:** Sendo uma rede social baseada em leitura massiva (*heavy-read*), a arquitetura do banco deve prever consultas eficientes e o uso de *Views* para otimizar o acesso restrito aos dados e proteger informações sensíveis.

4. Diagrama de casos de uso



5. Atores e funcionalidades

Na arquitetura de segurança do sistema, temos três atores principais:

- **Usuário comum:** vai poder enviar posts, visualizar o feed de postagens global e editar o conteúdo exclusivo de suas próprias postagens
- **Moderador:** Atua na governança da comunidade. Pode visualizar todas as publicações excluir posts que infrinjam as diretrizes da plataforma e atualizar o status das contas dos usuários (como aplicar suspensões ou banimentos)
- **Administrador (DBA):** Possui acesso irrestrito, sendo capaz de gerenciar todas as configurações do sistema realizar manutenção nas tabelas e administrar as contas de moderadores

6. Projeto de segurança e níveis de privilégio

O projeto de segurança definirá *Roles* (papéis) diretamente no banco de dados para segmentar o acesso, justificados pela necessidade da organização

- **role_usuario:** Receberá privilégios limitados a INSERT (para criar posts) e SELECT (para leitura). O privilégio de UPDATE será restrito via *Views* atualizáveis para garantir que o usuário só consiga editar linhas onde o `id_autor` seja igual ao seu próprio ID. O comando DELETE será expressamente negado, mantendo a integridade histórica (o usuário pode editar ou inativar via aplicação, mas o registro no banco permanece para auditoria).
- **role_moderador:** Terá permissões amplas de SELECT para monitoramento. Receberá o privilégio de DELETE na tabela de publicações, permitindo a exclusão de conteúdo irregular. Terá permissão de UPDATE apenas em colunas específicas da tabela de usuários (como a coluna `status_conta`), não podendo alterar dados pessoais como e-mail ou senha.
- **role_admin:** Possuirá os privilégios totais de manipulação de dados (DML) e definição de dados (DDL)

7. Tabelas, Acessos e Justificativas

Abaixo, temos o mapeamento de tabelas e como se dará o acesso de acordo com cada perfil:

- **Tabela Usuarios (id, nome, email, senha_hash, status_conta)**
 - **USUÁRIO COMUM:** Tem acesso de leitura (SELECT) apenas aos campos públicos (como nome) de outros usuários via uma View. Pode fazer UPDATE apenas na sua própria linha.
Justificativa: Proteção de dados sensíveis; um usuário jamais pode visualizar o e-mail ou a senha criptografada de terceiros.
 - **MODERADOR:** Tem SELECT estendido e UPDATE restrito à coluna `status_conta`. Sem acesso à `senha_hash`.

Justificativa: Necessário para auditar contas e aplicar punições, preservando o sigilo das credenciais.

- **Tabela Posts (id, id_autor, conteudo, data_publicacao)**

- USUÁRIO COMUM: INSERT para publicar, SELECT para carregar o feed e UPDATE condicional (via View) para alterar os próprios textos.

Justificativa: Reflete as funcionalidades essenciais de uso da rede social.

- MODERADOR: SELECT global e DELETE.

Justificativa: A moderação exige poder remover conteúdo indevido. O moderador **não** recebe UPDATE aqui, garantindo que ele não possa falsificar ou alterar as palavras originais escritas por um usuário.

- **Tabela Logs_Auditoria_Moderacao (id, id_moderador, acao_realizada, data)**

- USUÁRIO COMUM: Sem acesso (DENY).

Justificativa: São dados internos administrativos.

- MODERADOR: Permissão estrita de INSERT.

Justificativa: O moderador deve registrar suas ações ao excluir um post ou suspender alguém, mas a permissão de UPDATE ou DELETE nesta tabela é negada para evitar que ele encubra ou apague os rastros de ações abusivas.

O Administrador sempre terá acesso total, justificado pela necessidade de gerenciar o servidor, realizar backups e alterar a estrutura das tabelas.

8. Etapa Experimental (Planejamento Prático)

Para comprovar que os requisitos de segurança foram alcançados de forma prática, a etapa experimental consistirá em scripts SQL executados no console:

1. Serão criadas as contas no SGBD e as Roles pertinentes
2. Serão concedidos os privilégios (GRANT) estabelecidos nos itens 4 e 5

3. Teste de violação 1: Logar no banco com as credenciais de um usuário comum e tentar executar um DROP TABLE ou um DELETE FROM Posts. O SGBD deverá bloquear a ação com erro de permissão negada.
4. Teste de violação 2: Logar como moderador e tentar um UPDATE Posts SET conteúdo = 'fraude'. A ação deve ser rejeitada, comprovando que o isolamento de funções protege a veracidade do conteúdo da plataforma.

9. Desenvolvimento prático do sistema

Após a etapa inicial de planejamento da modelagem e segurança do banco de dados, o projeto Trecho foi desenvolvido de forma prática como uma aplicação web utilizando Flask, SQLAlchemy, PyMySQL e MySQL 8. A ideia principal continuou sendo a mesma definida no início do trabalho: criar uma rede social de microblogging, com posts curtos, interação entre usuários e controle de acesso por diferentes perfis.

Durante o desenvolvimento, o sistema deixou de ser apenas um planejamento de banco de dados e passou a contar com uma interface funcional, permitindo testar na prática os requisitos definidos anteriormente. Foram implementadas telas de login, feed, perfil, criação e edição de posts, comentários, curtidas, denúncias, notificações, moderação, relatórios e administração.

A aplicação foi organizada seguindo uma estrutura modular no Flask, separando as rotas por responsabilidade. Dessa forma, as funcionalidades de autenticação, posts, moderação, relatórios, administração, perfil e notificações ficaram divididas em arquivos diferentes, facilitando a manutenção e deixando o projeto mais organizado.

A comunicação com o banco de dados foi feita com SQLAlchemy e PyMySQL. Mesmo assim, uma parte importante das regras de negócio também foi mantida diretamente no banco por meio de procedures, views, roles e permissões. Isso foi feito para atender ao objetivo principal do trabalho, que é demonstrar que a segurança e a integridade dos dados

não dependem apenas da aplicação, mas também da própria estrutura do SGBD.

10. Estrutura implementada no banco de dados

O banco de dados recebeu o nome de rede_trecho e foi estruturado para armazenar as principais informações da rede social. As tabelas principais criadas foram as seguintes:

- perfis
- usuarios
- posts
- comentários
- curtidas
- seguidores
- denuncias
- logs_moderacao
- logs_login
- bloqueios
- notificacoes

A tabela perfis armazena os tipos de usuários do sistema. Foram definidos cinco perfis: visitante, usuario, moderador, analista e admin. Esses perfis são usados tanto pela aplicação quanto pelo banco de dados para controlar quais telas, ações e dados cada usuário pode acessar.

A tabela usuarios armazena as contas cadastradas no sistema, contendo informações como nome, username, email, senha criptografada, perfil, status da conta, data de cadastro e campos adicionais de perfil. Com as melhorias implementadas, essa tabela também passou a armazenar bio, foto de perfil, capa de perfil, data de término da suspensão e motivo da punição.

A tabela posts armazena as publicações feitas pelos usuários. Cada post possui um autor, conteúdo, status, data de publicação e data de atualização. Em vez de apagar fisicamente todos os registros em determinadas situações, o sistema utiliza status como ativo, oculto ou removido, mantendo o histórico para fins de auditoria.

A tabela comentarios permite que usuários respondam aos posts. Assim como os posts, os comentários possuem controle de status, permitindo a remoção lógica quando necessário.

A tabela curtidas registra as interações de curtida dos usuários nos posts. Foi implementada uma regra para evitar curtidas duplicadas do mesmo usuário no mesmo post.

A tabela denuncias registra as denúncias feitas por usuários contra publicações. Cada denúncia possui o post denunciado, o denunciante, o motivo, o status da denúncia e a data de registro.

A tabela logs_moderacao registra ações realizadas por moderadores e administradores, como remoção de posts, suspensão, banimento, reativação e alteração de perfil. Essa tabela é importante porque garante rastreabilidade das ações sensíveis realizadas dentro do sistema.

A tabela logs_login registra tentativas de login, permitindo acompanhar acessos bem-sucedidos ou falhos.

A tabela bloqueios armazena relações de bloqueio entre usuários. Quando um usuário bloqueia outro, o sistema impede determinadas interações e oculta conteúdos relacionados conforme a regra aplicada.

A tabela notificacoes foi criada para registrar avisos internos, como comentários recebidos em publicações. Essa funcionalidade torna o sistema mais próximo de uma rede social real.

11. Perfis de acesso implementados

Inicialmente, o projeto previa três perfis principais: usuário comum, moderador e administrador. Durante o desenvolvimento, a estrutura foi expandida para cinco perfis, permitindo uma separação mais completa das responsabilidades no sistema.

O perfil visitante representa o usuário não autenticado ou com acesso extremamente limitado. Esse perfil pode visualizar informações públicas, mas não pode criar posts, comentar, curtir, denunciar ou acessar áreas internas.

O perfil usuario representa o usuário comum da rede social. Esse perfil pode visualizar o feed, criar posts, editar seus próprios posts, curtir,

comentar, denunciar publicações, bloquear usuários e editar seu próprio perfil.

O perfil moderador representa o usuário responsável pela governança da comunidade. Ele pode acessar o painel de moderação, visualizar denúncias, remover posts denunciados, suspender usuários comuns e banir usuários comuns e remover punições de usuários comuns. O moderador não possui permissão para alterar a estrutura do banco, promover usuários ou punir administradores.

O perfil analista foi criado para representar um usuário que precisa visualizar dados gerenciais e relatórios, mas sem interferir diretamente nas interações sociais ou na moderação. Esse perfil pode acessar a tela de relatórios, mas não pode postar, comentar, moderar denúncias ou acessar a administração.

O perfil admin representa o administrador geral do sistema. Ele possui acesso à tela administrativa, pode visualizar usuários, logs, relatórios e também realizar ações mais sensíveis, como reativar contas, alterar perfis de usuários, promover usuários a moderadores e administrar a estrutura de permissões.

12. Funcionalidades sociais implementadas

O sistema passou a contar com um feed principal onde os usuários visualizam os posts ativos. Usuários autenticados com permissão podem criar novos posts diretamente pelo feed. O conteúdo dos posts é limitado, seguindo a proposta de uma rede social de microblogging.

Também foi implementada a edição de posts próprios. Um usuário comum pode alterar somente os posts que ele mesmo criou, respeitando o requisito de edição restrita definido no planejamento inicial. Essa regra é importante porque impede que um usuário modifique publicações de terceiros.

As curtidas foram implementadas como forma simples de interação. O sistema registra a curtida no banco de dados e também permite identificar se o usuário já curtiu determinado post.

Os comentários foram implementados para permitir respostas aos posts. Essa funcionalidade aumentou a interação entre usuários e também

exigiu regras adicionais no banco, já que usuários suspensos ou banidos não devem conseguir comentar.

A denúncia de posts também foi implementada. Quando um usuário denuncia uma publicação, essa denúncia fica registrada no banco e aparece para os moderadores no painel de moderação. Essa funcionalidade é essencial para conectar a participação dos usuários comuns com o fluxo de governança da plataforma.

Foi adicionada ainda a funcionalidade de bloqueio de usuários. O bloqueio impede determinadas interações entre dois usuários e representa uma camada adicional de controle social dentro da plataforma.

13. Perfil do usuário e personalização

Além das funcionalidades básicas de posts, o sistema recebeu uma tela de perfil para cada usuário. Nessa tela é possível visualizar informações públicas, como nome, username, bio e publicações feitas pelo usuário.

O próprio usuário pode editar sua bio, foto de perfil e capa. Para isso, foi implementado um modal de edição de perfil e um recorte simples de imagem, permitindo ajustar foto e capa antes de salvar. Essas informações são armazenadas no banco de dados nos campos bio, foto_perfil_url e capa_perfil_url.

A tela de perfil também foi integrada às ações sociais e administrativas. Usuários comuns podem bloquear outros usuários. Moderadores e administradores podem acessar um menu de ações no perfil de outro usuário, permitindo suspender, banir ou reativar contas, conforme o nível de permissão.

No caso do administrador, também foi implementada a alteração de perfil de usuários. Assim, um administrador pode promover um usuário comum para moderador, transformar um usuário em analista ou ajustar a hierarquia conforme a necessidade do sistema.

14. Moderação de conteúdo

A moderação foi uma das partes principais do projeto. O sistema possui uma tela específica de moderação, onde moderadores e administradores conseguem visualizar denúncias realizadas pelos usuários.

Quando um post é denunciado, ele aparece no painel de moderação com informações sobre o conteúdo, autor, denunciante, motivo da denúncia e status. A partir desse painel, o moderador pode remover o post ou aplicar punições ao autor.

A remoção de post não foi tratada apenas como uma exclusão visual. Ela altera o status da publicação no banco, permitindo manter o registro para auditoria. Essa abordagem é mais adequada do que simplesmente apagar o dado, pois mantém a rastreabilidade da ação.

Também foram implementadas ações de suspensão e banimento. A suspensão é uma punição temporária: o usuário suspenso consegue entrar no sistema e visualizar conteúdos, mas não pode publicar nem comentar. Já o banimento bloqueia o login do usuário, impedindo o acesso à aplicação.

A diferença entre suspensão e banimento foi importante para tornar o sistema mais realista. Nem toda violação precisa resultar em bloqueio total da conta. Em alguns casos, a suspensão temporária é suficiente para restringir apenas as ações de publicação e comentário.

15. Regras de suspensão, banimento e reativação

A conta suspensa continua podendo acessar o sistema. O usuário suspenso pode visualizar o feed, acessar o perfil, curtir posts, denunciar publicações, bloquear usuários e editar sua própria bio. Porém, ele não pode criar novos posts nem comentar.

Essa regra foi implementada tanto na interface quanto no backend. Na interface, os campos de postagem e comentário ficam desativados e exibem um aviso de suspensão. No backend, antes de chamar as procedures de criação de post ou comentário, o sistema verifica o status da conta e bloqueia a ação caso o usuário esteja suspenso ou banido.

A conta banida possui uma regra mais rígida. O usuário banido não consegue realizar login. Ao tentar entrar, o sistema exibe um aviso informando que a conta está banida.

Também foi implementada a reativação de usuários. Quando uma conta está suspensa, o sistema mostra a opção de remover suspensão. Quando uma conta está banida, o sistema mostra a opção de reativar usuário. Em

ambos os casos, o status da conta volta para ativo, mas a justificativa registrada no log é diferente, deixando claro se a ação foi uma remoção de suspensão ou uma reativação após banimento.

16. Views implementadas

Foram utilizadas views para simplificar consultas e proteger dados sensíveis. As views permitem exibir apenas as informações necessárias para cada contexto, evitando que a aplicação precise acessar diretamente todas as colunas das tabelas.

A view de feed público organiza os posts ativos com informações do autor, quantidade de curtidas e comentários. Essa view facilita a consulta do feed e permite retornar dados prontos para a interface.

A view de usuários públicos exibe apenas dados que podem ser vistos por outros usuários, como nome e username, sem expor senha, hash ou informações sensíveis.

A view de denúncias para moderação organiza os dados necessários para o painel dos moderadores, unindo informações de denúncias, posts, autores e denunciadores. Essa view foi ajustada para incluir também informações do perfil do autor, permitindo que a interface respeite regras de hierarquia.

A view de relatórios de engajamento apresenta dados agregados sobre posts, curtidas e comentários, auxiliando o perfil analista e o administrador na visualização de métricas do sistema.

A view de logs administrativos permite consultar registros importantes sem expor diretamente a estrutura completa das tabelas de log para perfis que não deveriam ter acesso total ao banco.

17. Procedures implementadas

As procedures foram utilizadas para centralizar regras importantes diretamente no banco de dados. Isso atende ao requisito de que a segurança e as regras de negócio não devem ficar apenas na aplicação. A procedure `sp_criar_post` é responsável por criar publicações. Ela valida se o usuário existe, se está ativo e se o conteúdo não está vazio. Dessa

forma, mesmo que alguém tente inserir um post fora da interface, o banco continua protegendo a regra.

A procedure `sp_editar_post` permite editar uma publicação, mas apenas se o usuário for o autor do post. Essa regra reforça o requisito de edição restrita.

A procedure `sp_denunciar_post` registra denúncias contra publicações. Ela valida se o post existe, se o denunciante existe e se o motivo da denúncia foi informado.

A procedure `sp_moderar_post` permite ações de moderação sobre publicações, como remover ou ocultar posts. Essa procedure também registra a ação em logs de moderação, garantindo auditoria.

A procedure `sp_alterar_status_usuario` permite alterar o status de uma conta para ativo, suspenso ou banido. Ela é usada em ações de suspensão, banimento e reativação. Essa procedure também registra a justificativa da ação.

A procedure `sp_comentar_post` permite inserir comentários em publicações. Ela valida se o usuário está ativo, se o post existe, se o post está ativo, se o comentário não está vazio e se não existe bloqueio entre os usuários envolvidos.

A procedure `sp_bloquear_usuario` registra bloqueios entre usuários. Ela impede que um usuário bloqueie a si mesmo e impede ações inválidas envolvendo contas inexistentes.

A procedure `sp_atualizar_perfil_usuario` permite atualizar bio, foto e capa do usuário. Ela impede que contas banidas editem o perfil.

A procedure `sp_alterar_perfil_usuario` permite que um administrador altere o perfil de outro usuário. Essa procedure valida se o executor é realmente administrador, se o usuário alvo existe, se o novo perfil é válido e se a justificativa foi informada.

18. Controle de permissões no banco de dados

O projeto também inclui um script de roles e permissões no banco de dados. O objetivo é representar no SGBD a separação de responsabilidades entre os perfis do sistema.

O usuário comum possui permissões limitadas para executar ações relacionadas ao uso normal da rede social, como consultar dados permitidos e interagir com posts através de procedures.

O moderador possui permissões voltadas para leitura de dados de moderação e execução de procedures responsáveis por remover posts, suspender ou banir usuários, respeitando as regras de hierarquia.

O analista possui acesso voltado para relatórios e consultas agregadas. Esse perfil não deve manipular posts, usuários ou ações de moderação.

O administrador possui permissões mais amplas, incluindo acesso a dados administrativos, logs, relatórios e execução de procedures sensíveis.

Essa separação ajuda a demonstrar que o sistema não depende apenas de verificações na interface. Mesmo que algum usuário tente acessar diretamente o banco com credenciais limitadas, as permissões impedem ações indevidas.

19. Importação do banco e tratamento de acentos

Durante o desenvolvimento, também foi necessário resolver problemas relacionados à codificação de caracteres, principalmente ao alternar modificações entre Windows e Linux. Alguns textos com acento apareciam incorretamente após reimportar o banco.

Para corrigir isso, o projeto passou a utilizar utf8mb4 na importação dos scripts SQL. O acesso ao MySQL deve ser feito com o parâmetro `--default-character-set=utf8mb4`, e o arquivo `run.sql` inicia com o comando `SET NAMES utf8mb4`.

Também foi criado o arquivo `run.sql` para simplificar a importação do banco. Em vez de executar manualmente todos os scripts, basta abrir o MySQL na raiz do projeto e executar `SOURCE run.sql`. Esse arquivo importa o schema, dados iniciais, views, procedures, permissões, consultas e melhorias sociais.

Além disso, foi criado o script `07_melhorias_sociais.sql`, responsável por garantir campos e estruturas adicionadas posteriormente, como bio, foto de perfil, capa, suspensão, motivo da punição, bloqueios, comentários, notificações e procedures complementares.

20. README e documentação de execução

O README do projeto foi atualizado para documentar o processo de instalação e execução. Ele passou a conter instruções para clonar o repositório, criar e ativar ambiente virtual, instalar dependências, configurar o arquivo .env, verificar o serviço do MySQL, importar o banco com run.sql, atualizar as senhas dos usuários de teste e executar o projeto.

Também foram adicionadas observações sobre acentos, explicando a importância de salvar os arquivos SQL como UTF-8 e abrir o MySQL com utf8mb4.

O README também apresenta os usuários de teste, os perfis do sistema, a política de suspensão e banimento e um resumo de comandos necessários.

21. Testes automatizados com Selenium

Para validar o funcionamento do sistema, foi criado um fluxo automatizado de testes com Selenium e pytest. O objetivo foi simular o uso real da plataforma por diferentes tipos de usuários.

O teste começa com um convidado acessando o feed. Em seguida, esse convidado tenta interagir com o sistema e recebe o modal de aviso para entrar ou criar uma conta. Depois, é testado o login com senha incorreta e, em seguida, o login correto com um usuário comum.

Com o usuário comum, o teste cria um post, edita o post, acessa notificações, acessa o perfil, edita a bio, comenta em um post, curte uma publicação, denuncia um post, bloqueia um usuário, remove o bloqueio pelo banco para continuar o fluxo e realiza uma nova denúncia.

Depois disso, o fluxo continua com o moderador. O moderador acessa a tela de moderação, remove um post denunciado, acessa o perfil de uma usuária e aplica suspensão. Em seguida, o teste valida que a usuária suspensa consegue entrar no sistema, mas não consegue postar nem comentar.

Depois, o moderador realiza o banimento da usuária. O teste então confirma que a conta banida não consegue realizar login.

Também foi testado o perfil analista. O analista acessa os relatórios, mas não deve conseguir postar, moderar ou acessar a área administrativa. Essa validação é importante para comprovar a separação entre visualização gerencial e ações de moderação.

Por fim, o administrador acessa a tela de administração, reativa a usuária banida, promove essa usuária a moderadora e o teste valida que a conta promovida consegue acessar funcionalidades de moderação. Esse fluxo comprova que a alteração de hierarquia funciona corretamente.

22. Evidências geradas

Durante a execução dos testes, o Selenium gera capturas de tela em diferentes etapas do fluxo. Essas imagens podem ser utilizadas como evidências no relatório, mostrando visualmente que cada ação foi realizada.

Entre as evidências previstas estão: feed público, modal de login, tentativa de login inválida, criação e edição de post, tela de notificações, edição de perfil, comentário, curtida, denúncia, bloqueio, painel de moderação, remoção de post, suspensão, conta suspensa sem permissão de postar ou comentar, banimento, bloqueio de login para conta banida, relatórios do analista, administração do sistema e promoção de usuário.

Essas evidências complementam os testes SQL e demonstram que as regras do banco também estão refletidas na interface da aplicação.

23. Resultados obtidos

Com a implementação realizada, o sistema passou a atender aos principais requisitos funcionais e não funcionais definidos inicialmente. O usuário comum consegue utilizar a rede social, criar conteúdo e interagir com outros usuários. O moderador consegue atuar na governança da comunidade, removendo posts e aplicando punições. O analista consegue acessar informações de relatório sem interferir na operação da plataforma. O administrador consegue gerenciar usuários, status, hierarquia e dados administrativos.

As regras de segurança foram aplicadas em diferentes camadas. Na interface, botões e campos são exibidos ou bloqueados conforme o perfil

do usuário. No backend, as rotas verificam permissões antes de executar ações sensíveis. No banco de dados, views, roles e procedures reforçam a proteção dos dados e a separação das responsabilidades.

A utilização de logs também reforçou a rastreabilidade, permitindo registrar ações de moderação, login e alterações administrativas. Isso atende ao requisito de auditoria e evita que ações sensíveis sejam realizadas sem registro.

24. Conclusão

O desenvolvimento do Trecho permitiu transformar o planejamento inicial de segurança e modelagem em um sistema funcional. A aplicação implementada demonstra, na prática, como uma rede social de microblogging pode utilizar banco de dados relacional, controle de acesso, procedures, views, logs e roles para manter integridade, segurança e rastreabilidade.

O projeto também mostrou a importância de não concentrar toda a regra de negócio apenas na aplicação. Mesmo que o Flask controle as telas e os fluxos de navegação, o banco de dados continua tendo papel essencial na validação das ações, na proteção das informações e no registro das operações administrativas.

Com os testes automatizados e as evidências geradas, foi possível validar os principais fluxos do sistema, incluindo uso comum, moderação, relatórios, administração, suspensão, banimento, reativação e alteração de perfil. Dessa forma, o sistema desenvolvido conclui a segunda parte do TDE, demonstrando tanto a modelagem quanto a administração e segurança do banco de dados em um cenário prático.